

RELATÓRIO TÉCNICO

Script de Preparação e Estruturação de Dados para Otimização das Rotas das Equipes de Fiscalização do IPEM-SP

Projeto Integrador – API3 (Pesquisa Operacional) – LOGTEAM 3

FATEC Professor Jessen Vidal – Tecnologia em Logística

1. Objetivo do Script

Este documento descreve, em detalhe e na ordem em que aparecem no código, todas as etapas realizadas pelo script Python (executado em ambiente Google Colab) responsável por preparar a base de dados de fiscalizações do IPEM-SP para a etapa de otimização de rotas do Projeto Integrador. O script recebe uma planilha bruta (CSV) com os registros de fiscalização, padroniza endereços, identifica setores (bairros), monta identificadores únicos, reconstrói a sequência de visitas de cada equipe ao longo do dia e gera, para cada trecho percorrido, um link pronto do Google Maps com a rota, incluindo o trajeto de ida a partir da Sede e o retorno à Sede ao final do expediente.

O resultado final é salvo em dois formatos (.csv e .xlsx), prontos para serem usados nas etapas seguintes de análise e modelagem de Pesquisa Operacional (cálculo de distâncias, tempos e otimização das rotas).

2. Ambiente e Bibliotecas Utilizadas

Importações e montagem do Google Drive

```
import os
import re
import urllib.parse
import numpy as np
import pandas as pd
from google.colab import drive
```

```
drive.mount('/content/drive')
```

O que faz: Importa as bibliotecas necessárias: os (caminhos de arquivo), re (expressões regulares para limpeza de texto), urllib.parse (codificação de endereços para URLs), numpy e pandas (manipulação de dados tabulares) e o módulo drive do Google Colab. Em seguida, monta o Google Drive do usuário no ambiente do Colab, permitindo ler e gravar arquivos diretamente na pasta do projeto (LOG3/API3/LOGTEAM 3).

Definição dos caminhos e do ponto fixo (Sede)

```
input_path = '/content/drive/My Drive/LOG3/API3/LOGTEAM 3/data v1.csv'
output_dir = '/content/drive/My Drive/LOG3/API3/LOGTEAM 3/'
```

```
SEDE_ENDERECO = (
    'R. SEBASTIÃO HUMEL, 759 - CENTRO, SÃO JOSÉ DOS CAMPOS - SP, 12210-200'
)
```

O que faz: Define o caminho do arquivo de entrada (planilha bruta 'data v1.csv') e a pasta de saída onde os resultados serão salvos. Também define uma constante com o endereço fixo da Sede do IPEM-SP, que será usada como ponto de partida da primeira visita do dia de cada fiscal e como ponto de retorno ao final do roteiro.

Leitura da planilha e preservação da ordem original

```
df = pd.read_csv(input_path, encoding='latin1')
df['LINHA_ORIGINAL_PLANILHA'] = df.index + 1
```

O que faz: Lê o arquivo CSV de entrada para um DataFrame do pandas, usando codificação 'latin1' (comum em planilhas exportadas em português com acentuação). Cria a coluna LINHA_ORIGINAL_PLANILHA, que guarda a posição de cada registro na planilha original — essa referência é essencial porque, mais adiante, o script reordena e reagrupa os dados, e essa coluna garante que a ordem física original possa ser restaurada ao final.

2. Padronização e Normalização de Endereços

Função clean_address()

```
def clean_address(addr):
    if pd.isna(addr):
        return ""
    addr = str(addr).upper().strip()
    addr = re.sub(r'\s+', ' ', addr)
    addr = re.sub(r'\bAVENIDA\b|\bAV\b', 'AV', addr)
    addr = re.sub(r'\bRUA\b|\bR\b', 'RUA', addr)
    addr = re.sub(r'\bALAMEDA\b|\bAL\b', 'AL', addr)
    addr = re.sub(r'\bDOUTOR\b|\bDR\b', 'DR', addr)
    return addr
```

O que faz: Define uma função que padroniza o texto de um endereço: trata valores vazios (retorna string vazia), converte tudo para maiúsculas e remove espaços nas pontas, colapsa múltiplos espaços em um só, e uniformiza abreviações comuns de logradouro (AVENIDA/AV. → AV; RUA/R. → RUA; ALAMEDA/AL. → AL; DOUTOR/DR. → DR). Isso evita que o mesmo endereço apareça escrito de formas diferentes (ex.: 'Av.' e 'AVENIDA'), o que prejudicaria o agrupamento e a geolocalização posteriores.

Aplicação da padronização

```
df['ENDERECO_PADRONIZADO'] = df['ENDERECO_COMPLETO'].apply(clean_address)
```

O que faz: Aplica a função clean_address a cada valor da coluna ENDERECO_COMPLETO e armazena o resultado em uma nova coluna, ENDERECO_PADRONIZADO, que será usada em todas as etapas seguintes do script (agrupamento de paradas, geração de rotas, etc.).

3. Extração de Setor (Bairro) e Criação de IDs

Função extract_bairro()

```
def extract_bairro(addr):
    if pd.isna(addr):
        return 'CENTRO'
    parts = [p.strip() for p in str(addr).split('-') if p.strip()]
    if len(parts) >= 2:
        bairro = parts[1].upper().strip()
        bairro = bairro.split('CEP')[0].strip()
```

```
    return bairro if bairro else 'CENTRO'
return 'CENTRO'
```

O que faz: Define uma função que extrai o bairro (setor) a partir do texto completo do endereço, dividindo a string pelo caractere '-'. Assume-se que o segundo segmento (após o primeiro '-') corresponde ao bairro. Remove qualquer trecho referente ao CEP que porventura esteja junto. Caso o endereço esteja vazio ou não siga o padrão esperado, o bairro é definido como 'CENTRO' por padrão.

Aplicação da extração e criação dos IDs de setor

```
df['NOME_SETOR'] = df['ENDERECO_COMPLETO'].apply(extract_bairro)
```

```
setores_unicos = sorted(df['NOME_SETOR'].unique())
setor_to_id = {
    setor: f'SET-{str(i+1).zfill(4)}' for i, setor in enumerate(setores_unicos)
}
```

```
df['ID_SETOR'] = df['NOME_SETOR'].map(setor_to_id)
```

O que faz: Cria a coluna NOME_SETOR aplicando extract_bairro a cada linha. Em seguida, obtém a lista de bairros únicos em ordem alfabética e monta um dicionário que associa cada bairro a um identificador alfanumérico padronizado no formato SET-0001, SET-0002, etc. Por fim, cria a coluna ID_SETOR, mapeando cada registro ao ID do seu respectivo setor. Esse identificador facilita o agrupamento e a referência aos setores nas etapas de otimização.

Mapeamento dos setores de atuação por fiscal

```
setores_fiscal_map = (
    df.groupby('FISCAL')['ID_SETOR']
    .apply(lambda x: ', '.join(sorted(x.unique())))
    .to_dict()
)
```

```
nomes_setores_fiscal_map = (
    df.groupby('FISCAL')['NOME_SETOR']
    .apply(lambda x: ', '.join(sorted(x.unique())))
    .to_dict()
)
```

```
df['LISTA_IDS_SETORES_ATUACAO_FISCAL'] = df['FISCAL'].map(setores_fiscal_map)
df['LISTA_NOMES_SETORES_ATUACAO_FISCAL'] = df['FISCAL'].map(
    nomes_setores_fiscal_map
)
```

O que faz: Agrupa os registros por fiscal e reúne, para cada um, a lista (sem repetição e em ordem alfabética) de todos os IDs de setor e de todos os nomes de setor em que aquele fiscal atuou. Essas listas são então mapeadas de volta para o DataFrame original nas colunas LISTA_IDS_SETORES_ATUACAO_FISCAL e LISTA_NOMES_SETORES_ATUACAO_FISCAL, dando uma visão consolidada da abrangência geográfica de atuação de cada fiscal.

3.1 Identificadores de Equipe, Tipo de Fiscalização e Registro

ID_EQUIPE, ID_TIPO_FISCALIZACAO e ID_REGISTRO_FISCALIZACAO

```
df['ID_EQUIPE'] = (  
    'EQP-'  
    + df['IDFISCAL'].astype(str)  
    + '-'  
    + df['IDMOTORISTA'].fillna(0).astype(int).astype(str)  
)  
df['ID_TIPO_FISCALIZACAO'] = (  
    'FISC-'  
    + df['SERVIÇO']  
    .astype('category')  
    .cat.codes.add(1)  
    .astype(str)  
    .str.zfill(3)  
)  
df['ID_REGISTRO_FISCALIZACAO'] = [  
    'FISC-REC-' + str(i).zfill(6) for i in range(1, len(df) + 1)  
]
```

O que faz: Cria três identificadores derivados: (1) ID_EQUIPE, combinando o ID do fiscal e o ID do motorista no formato EQP-<fiscal>-<motorista>, tratando motoristas não informados como 0; (2) ID_TIPO_FISCALIZACAO, que converte a coluna de serviço em uma categoria numérica sequencial e monta um código no formato FISC-XXX; e (3) ID_REGISTRO_FISCALIZACAO, um identificador sequencial único para cada linha do arquivo, no formato FISC-REC-XXXXXX, servindo como chave primária do registro.

4. Reconstrução das Rotas na Sequência Física Real

Detecção de mudança de ponto de parada (MUDANCA_PONTO)

```
df['MUDANCA_PONTO'] = (  
    (df['IDFISCAL'] != df['IDFISCAL'].shift(1))  
    | (df['ENDERECO_PADRONIZADO'] != df['ENDERECO_PADRONIZADO'].shift(1))  
    | (df['DATAVERIFICACAO'] != df['DATAVERIFICACAO'].shift(1))  
)  
.astype(int).cumsum()
```

O que faz: Compara cada linha com a linha imediatamente anterior (shift(1)) em três critérios: mudança de fiscal, mudança de endereço padronizado ou mudança de data de verificação. Sempre que qualquer um desses três valores muda em relação à linha anterior, marca o início de uma nova 'parada física'. A soma acumulada (cumsum) gera um identificador crescente (MUDANCA_PONTO) que agrupa todas as linhas pertencentes à mesma parada — importante porque um mesmo endereço pode ter várias linhas (um item fiscalizado por linha), mas representa fisicamente uma única visita.

Agrupamento sequencial das paradas

```
paradas = (  

```

```

df.groupby('MUDANCA_PONTO')
.agg(
    DATAVERIFICACAO=('DATAVERIFICACAO', 'first'),
    IDFISCAL=('IDFISCAL', 'first'),
    FISCAL=('FISCAL', 'first'),
    ID_EQUIPE=('ID_EQUIPE', 'first'),
    ENDERECO_PADRONIZADO=('ENDERECO_PADRONIZADO', 'first'),
    LINHA_INICIAL=('LINHA_ORIGINAL_PLANILHA', 'min'),
    QTD_ITENS=('CODITEM', 'count'),
)
.reset_index()
.sort_values(by='LINHA_INICIAL')
)

```

O que faz: Agrupa o DataFrame pelo identificador de parada (MUDANCA_PONTO) criado na etapa anterior e resume cada grupo em uma única linha, trazendo a data, o fiscal, a equipe e o endereço daquela parada (usando o primeiro valor do grupo, já que são iguais dentro do grupo), a linha original mais antiga (para preservar a ordem cronológica) e a quantidade de itens fiscalizados naquela parada (contagem de CODITEM). O resultado, chamado paradas, é ordenado pela linha inicial, reconstituindo a sequência física real de visitas de cada fiscal.

Numeração da sequência de visitas do dia

```

paradas['SEQ_VISITA_DIA'] = (
    paradas.groupby(['DATAVERIFICACAO', 'IDFISCAL']).cumcount() + 1
)
paradas['TOTAL_VISITAS_DIA'] = paradas.groupby(['DATAVERIFICACAO', 'IDFISCAL'])[
    'SEQ_VISITA_DIA'
].transform('max')

```

O que faz: Para cada combinação de data e fiscal, numera as paradas em ordem crescente (1ª visita do dia, 2ª visita, etc.) na coluna SEQ_VISITA_DIA, e calcula também o total de visitas realizadas naquele dia por aquele fiscal (TOTAL_VISITAS_DIA). Essas duas colunas serão usadas para identificar a primeira visita (ponto de partida a partir da Sede) e a última visita (ponto de retorno à Sede) de cada dia.

Definição do ponto de origem de cada trecho

```

def definir_origem(row):
    if row['SEQ_VISITA_DIA'] == 1:
        return SEDE_ENDERECO
    else:
        idx = row.name
        return paradas.loc[idx - 1, 'ENDERECO_PADRONIZADO']

```

```

paradas['ENDERECO_ORIGEM_TRECHO'] = paradas.apply(definir_origem, axis=1)

```

O que faz: Define, para cada parada, qual é o endereço de origem do trecho percorrido até ali: se for a primeira visita do dia (SEQ_VISITA_DIA == 1), a origem é a Sede (SEDE_ENDERECO); caso contrário, a origem é o endereço da parada imediatamente anterior na tabela paradas. O resultado é armazenado em ENDERECO_ORIGEM_TRECHO.

Geração dos links de rota (Google Maps)

```
def make_gmaps_url(origem, destino):
    orig = urllib.parse.quote(str(origem))
    dest = urllib.parse.quote(str(destino))
    return f'https://www.google.com/maps/dir/?api=1&origin={orig}&destination={dest}&travelmode=driving'

paradas['LINK_ROTA_TRECHO_GOOGLE_MAPS'] = paradas.apply(
    lambda r: make_gmaps_url(
        r['ENDERECO_ORIGEM_TRECHO'], r['ENDERECO_PADRONIZADO']
    ),
    axis=1,
)
```

O que faz: Define uma função auxiliar que monta uma URL de rota do Google Maps (modo de direções, viagem de carro) entre um endereço de origem e um endereço de destino, codificando os caracteres especiais dos endereços para uso seguro em URL (`urllib.parse.quote`). Essa função é aplicada a cada parada, gerando a coluna `LINK_ROTA_TRECHO_GOOGLE_MAPS` com o link de navegação entre a origem do trecho e o endereço daquela parada.

Função gerar_retorno_sede() — link de retorno à Sede ao final do dia

```
def gerar_retorno_sede(row):
    if row['SEQ_VISITA_DIA'] == row['TOTAL_VISITAS_DIA']:
        return make_gmaps_url(row['ENDERECO_PADRONIZADO'], SEDE_ENDERECO)
    else:
        return ""

paradas['LINK_RETORNO_SEDE_GOOGLE_MAPS'] = paradas.apply(
    gerar_retorno_sede, axis=1
)
```

O que faz: Define a função `gerar_retorno_sede`, que recebe uma linha (parada) e verifica se ela corresponde à última visita do dia daquele fiscal (`SEQ_VISITA_DIA` igual a `TOTAL_VISITAS_DIA`). Se for a última visita, gera um link do Google Maps do endereço dessa parada de volta até a Sede; caso contrário, retorna uma string vazia (`""`). A função é então aplicada linha a linha (`apply` com `axis=1`) sobre a tabela `paradas`, preenchendo a coluna `LINK_RETORNO_SEDE_GOOGLE_MAPS`. Isso garante que cada dia de trabalho de cada fiscal termine com um trajeto de retorno explícito à Sede, sem gerar esse link nas paradas intermediárias.

5. Reincorporação dos Resultados à Base Original

Merge das informações de rota de volta para o DataFrame original

```
df = df.merge(
    paradas[[
        'MUDANCA_PONTO',
        'SEQ_VISITA_DIA',
```

```

'TOTAL_VISITAS_DIA',
'ENDERECO_ORIGEM_TRECHO',
'LINK_ROTA_TRECHO_GOOGLE_MAPS',
'LINK_RETORNO_SEDE_GOOGLE_MAPS',
],
on='MUDANCA_PONTO',
how='left',
)

```

O que faz: Realiza um merge (junção) do DataFrame original (df) com as colunas calculadas na tabela agregada de paradas (número da visita no dia, total de visitas, endereço de origem do trecho e os dois links de rota), usando MUDANCA_PONTO como chave de junção do tipo 'left'. Com isso, todas as linhas originais (inclusive as que representam itens individuais de uma mesma parada) recebem as informações de rota calculadas ao nível da parada.

Restauração da ordem física original

```

df = (
    df.sort_values(by='LINHA_ORIGINAL_PLANILHA')
    .drop(columns=['MUDANCA_PONTO'])
    .reset_index(drop=True)
)

```

```
df.head(5)
```

O que faz: Reordena o DataFrame de volta pela coluna LINHA_ORIGINAL_PLANILHA (definida logo no início do script), garantindo que a ordem final das linhas seja idêntica à da planilha de entrada. Remove a coluna auxiliar MUDANCA_PONTO, que não é mais necessária, e reindexa o DataFrame. A chamada df.head(5) exibe as cinco primeiras linhas do resultado como conferência visual dentro do notebook.

6. Exportação dos Resultados

Gravação em CSV e XLSX

```

csv_output = os.path.join(output_dir, 'data_v1_ETL_final.csv')
xlsx_output = os.path.join(output_dir, 'data_v1_ETL_Final.xlsx')

df.to_csv(csv_output, index=False, encoding='utf-8-sig')
df.to_excel(xlsx_output, index=False, engine='openpyxl')

print("Processamento concluído!")
print(f"Planilha salva em CSV: {csv_output}")
print(f"Planilha salva em XLSX: {xlsx_output}")

```

O que faz: Monta os caminhos completos dos arquivos de saída (data_v1_ETL_final.csv e data_v1_ETL_Final.xlsx) dentro da pasta do projeto no Google Drive. Salva o DataFrame final em CSV (codificação utf-8-sig, compatível com acentuação no Excel) e em XLSX (usando o motor openpyxl), ambos sem a coluna de índice do pandas. Por fim, imprime mensagens de confirmação no console informando que o processamento foi concluído e onde cada arquivo foi salvo.

7. Síntese do Fluxo de Processamento

Em resumo, o script segue a seguinte sequência lógica de transformação dos dados:

- Leitura da planilha bruta e preservação da ordem original das linhas.
- Padronização textual dos endereços (maiúsculas, espaços, abreviações de logradouro).
- Extração do bairro/setor de cada endereço e criação de IDs únicos de setor (SET-XXXX).
- Consolidação dos setores de atuação de cada fiscal.
- Criação de identificadores únicos de equipe (fiscal + motorista), tipo de fiscalização e registro.
- Detecção das paradas físicas reais (mudança de fiscal, endereço ou data) e agrupamento sequencial dessas paradas.
- Numeração da ordem de visita de cada fiscal em cada dia e identificação da primeira e da última parada do dia.
- Definição do endereço de origem de cada trecho (Sede na 1ª visita; parada anterior nas demais).
- Geração automática dos links de rota do Google Maps para cada trecho, com uma função dedicada (gerar_retorno_sede) para o link de retorno à Sede ao final do dia.
- Reincorporação dessas informações à base original, restauração da ordem física da planilha e exportação final em CSV e XLSX.

Essa base tratada e enriquecida com IDs, setores e rotas georreferenciadas é o insumo de entrada para a etapa seguinte do Projeto Integrador, referente à otimização (Pesquisa Operacional) do planejamento das equipes de fiscalização do IPEM-SP na região de São José dos Campos.